A Comparative Study of Chunking Strategies, Embedding Models and Retrieval Pipelines for RAG over Technical Documentation

Vinayak Kalal, Samvit Nandakishore, Sabbir Mondal, Rahul Shenoy
SRNs: PES2UG25AM809, PES2UG24AM144, PES2UG24AM141, PES2UG24AM129

***Abstract*—Retrieval-augmented generation (RAG) depends on how documents are split into chunks, how chunks are embedded and how retrieval is performed, yet these choices are often made by habit. We present a staged empirical study of nine chunking strategies, four open-source embedding models and three retrieval pipelines (dense, hybrid BM25 plus dense, and hybrid with a cross-encoder reranker) on Python library documentation. Retrieval is scored with chunk-size-independent span-overlap metrics against 85 LLM-drafted, evidence-grounded questions, and answer quality with an LLM judge. On this corpus most chunkers are statistically indistinguishable (best f_{ctx} 0.729; semantic chunking lowest at 0.634), the best embedder depends on the chunker, hybrid retrieval raises mean f_{ctx} from 0.720 to 0.799, and the gap between the best and worst chunker shrinks from 0.095 to 0.059 when a reranker is added. Truncating embeddings to 256 dimensions keeps 97% of quality with two-thirds of the storage. We distil the results into a decision matrix and a Config Advisor tool.**

***Index Terms*—*Retrieval-augmented generation, chunking, text embeddings, hybrid retrieval, reranking, RAG evaluation*

I. INTRODUCTION

A. Motivation

Retrieval-augmented generation (RAG) grounds a large language model in external documents by retrieving relevant passages at query time [1]. The quality of a RAG answer is bounded by the quality of retrieval: if the needed evidence is missing from the retrieved context, or buried among irrelevant text, the generator cannot recover [3]–[5]. Three engineering decisions determine what the generator sees: how documents are split into chunks, which embedding model represents the chunks, and how candidates are retrieved and ordered. Tutorials and framework defaults usually suggest a fixed chunk size with some overlap and a popular embedding model, and teams rarely test whether that default suits their own documents.

B. Problem and Gap

Prior work examines these decisions mostly one at a time. Studies of chunking compare strategies on a single dataset or domain [7]–[10]. Embedding benchmarks such as MTEB and BEIR score models on passages that are already segmented [18], [19]. Evaluations of RAG components seldom break results down by document type [25]. In the works we reviewed we found no study that varies chunking, embedding model and retrieval pipeline together using a metric that does not depend on chunk size, nor one that also reports what embedding compression costs in quality. A practitioner therefore cannot tell whether a published recommendation transfers to their own documents.

C. Objectives and Contributions

This paper addresses Problem Statement 6 of the course: a systematic comparison of chunking strategies and embedding models on RAG retrieval. We ask three research questions. **RQ1:** does the best chunking and embedding combination depend on the document and on the pipeline around it? **RQ2:** does a stronger retrieval pipeline reduce the sensitivity of results to the chunking choice? **RQ3:** how much quality is lost when embeddings are truncated or quantized? These map to three hypotheses: **H1**, structure-aware chunking outperforms fixed-size and semantic chunking on structured documents; **H2**, a reranker narrows the gap between chunking strategies; **H3**, truncated or binary embeddings lose little accuracy while cutting storage and latency. Our contributions are:

1) A span-based evaluation protocol in which gold evidence is stored as character offsets, so retrieval metrics are comparable across chunk sizes and need no LLM.
2) A staged experimental design that keeps the 9 × 4 × 3 grid of chunkers, embedders and retrieval modes tractable on a CPU-only machine.
3) Empirical results on technical documentation covering chunk strategy, embedding model, retrieval pipeline, answer quality and compression, with confidence intervals and a paired significance test.
4) A decision matrix and a Config Advisor application, backed by a single reproducible notebook in which the LLM provider can be switched between Gemini and OpenRouter.

D. Paper Organisation

Section II reviews related work and states the research gap. Section III describes the data and methodology, Section IV the implementation and Section V the experimental setup. Section VI presents the results, Section VII the decision matrix and advisor, and Section VIII relates the findings to the research questions and prior work. Section IX discusses limitations and future work and Section X concludes. The Appendix lists the prompts and fixed settings.

E. Relevance to Prompt Engineering

Although the subject is retrieval, prompt design is part of the method in three places: the prompts that draft evaluation questions with verifiable evidence, the constrained generation prompt that makes abstention detectable, and the judge prompt that scores answers in a single call. Section IV.C explains these design choices, and the Appendix gives the prompts in full. Because the generator is instructed to rely only on retrieved context, differences in answer quality can be attributed to what was retrieved and not to the model's own knowledge.

II. RELATED WORK

A. RAG Foundations and Failure Modes

Lewis et al. [1] combined a parametric generator with a retrieval memory, and dense retrievers such as DPR [2] made semantic passage retrieval practical. Gao et al. [3] group RAG systems into naive, advanced and modular designs, where advanced systems add query rewriting before retrieval and reranking after it. Barnett et al. [4] report seven failure points from engineering experience, several concerning evidence that is missing, ranked too low or not usable in the final context. Liu et al. [5] show that models use information less reliably when it sits in the middle of a long context, and Cuconasu et al. [6] find that the kind of irrelevant text in the retrieved context

changes accuracy in non-obvious ways. Together these results motivate measuring recall, precision and noise jointly.

B. Chunking

Fixed-size chunking with overlap is the framework default. Semantic chunking, which splits where neighbouring sentences diverge, is popular in practitioner material, but Qu et al. [7] report that its gains over fixed-size chunking are not consistent enough to justify the extra computation. Chen et al. [8] study retrieval granularity and show that indexing propositions can improve retrieval and downstream question answering at the price of LLM preprocessing. Yepes et al. [9] propose element-based chunking that follows the structure of financial reports and report gains over fixed-size baselines, and Finardi et al. [10] study how retriever, chunking and generator choices jointly affect RAG quality. Late chunking [11] and Contextual Retrieval [12] enrich chunk representations with document context, while parent-document (small-to-big) retrieval embeds small units but returns a larger parent section.

C. Embedding Models and Compression

Sentence-BERT [13] established bi-encoder sentence embeddings. Open models such as E5 [14], BGE [15], GTE [16] and Nomic Embed [17] are trained with large-scale contrastive learning. MTEB [18] and BEIR [19] average performance over many tasks and mostly evaluate pre-segmented passages, so they say little about behaviour after chunking or across document types. Matryoshka Representation Learning [20] trains vectors whose leading dimensions remain useful when truncated, and binary or scalar quantization can shrink indexes while keeping much of retrieval quality [21]. These results come from benchmark settings, not from combinations with chunking choices.

D. Retrieval Pipelines and Evaluation

BM25 [22] remains a strong sparse baseline for exact terms and identifiers. Reciprocal Rank Fusion (RRF) [23] merges sparse and dense rankings without training, and cross-encoder rerankers [24] re-score a short candidate list jointly with the query. Wang et al. [25] compare combinations of RAG components, and FAISS [26] provides efficient vector search. RAGAS [27] defines reference-free RAG metrics that rely on an LLM judge, which Zheng et al. [28] show agrees well with humans but has position, verbosity and self-enhancement biases. We avoid LLM judging for retrieval and keep it for answer-level metrics.

Table I compares twenty of the 28 works reviewed, grouped by theme, and states how each relates to this study. In almost every case the work varies one or two factors on one dataset, which is the gap this study addresses.

Work	Focus	Main finding	Relation to this study
Lewis et al. [1]	RAG formulation	Generator combined with a retrieval memory for knowledge-intensive tasks	Basis of our pipeline
Gao et al. [3]	RAG survey	Naive, advanced and modular RAG	Frames our dense, hybrid, reranked progression
Barnett et al. [4]	Failure points	Seven failure points, several in retrieval	Motivates retrieval-focused metrics
Qu et al. [7]	Semantic vs fixed chunking	Semantic gains not consistent enough for the cost	We compare it across embedders
Chen et al. [8]	Retrieval granularity	Proposition units can improve retrieval and QA	We test cheaper units and parent-child
Yepes et al. [9]	Element-based chunking	Structure-aware chunking helped on financial reports	We test it on API documentation
Finardi et al. [10]	Retriever, chunk, generator	Components interact and should be studied together	Supports our staged design
Günther et al. [11]	Late chunking	Embeds long text first, then pools chunk vectors	Future extension
Anthropic [12]	Contextual Retrieval	Prepends LLM-written context to each chunk	Future extension
Reimers and Gurevych [13]	Sentence-BERT	Established bi-encoder sentence embeddings	Basis of our embedding comparison
Wang et al. [14]	E5 embeddings	Weakly-supervised contrastive pre-training	e5-small is one of our models
Xiao et al. [15]	BGE embeddings	Open general-purpose embedding family	bge-small is one of our models
Kusupati et al. [20]	Matryoshka vectors	Leading dimensions stay useful when truncated	Basis of our truncation tests
Shakir et al. [21]	Binary and scalar quantization	Smaller indexes that keep much of the quality	Basis of our binary tests
Robertson and Zaragoza [22]	BM25	Strong sparse baseline for exact terms	Sparse half of hybrid mode
Cormack et al. [23]	Reciprocal Rank Fusion	Training-free merging of rankings	Our hybrid fusion
Nogueira and Cho [24]	Cross-encoder reranking	Joint query and passage scoring improves ordering	Our reranking stage
Wang et al. [25]	RAG best practices	Recommends balanced component combinations	General; we examine chunkers in detail
Es et al. [27]	RAGAS metrics	Reference-free RAG metrics using an LLM judge	We use span metrics for retrieval
Zheng et al. [28]	LLM-as-a-judge	High agreement with humans but known biases	We verify questions and cross-check judges

TABLE I
COMPARISON OF TWENTY KEY WORKS

E. Research Gap

Taken together, the reviewed work leaves five gaps that this study addresses.

1) *Limited document diversity.* Comparisons usually use one generic or single-domain corpus, so it is unclear whether a recommendation transfers between document types.
2) *Components studied in isolation.* Chunking, embedding model and retrieval pipeline are rarely varied together, and it is untested whether reranking reduces the importance of the chunking choice.
3) *Metrics tied to chunk boundaries.* Common retrieval metrics depend on how the corpus was chunked, which complicates fair comparison between chunkers of different sizes.
4) *Efficiency studied separately.* Truncation and quantization are evaluated on benchmarks and not jointly with chunking, latency and index size.
5) *No practitioner-ready output.* Findings are seldom turned into a decision aid mapping document type and priorities to a configuration.

III. DATASET AND METHODOLOGY

A. Corpus

We use eight pages of the Python standard-library documentation [29] (argparse, json, re, pathlib, itertools, collections, functools and logging), 70,791 words in total. HTML is converted to lightly structured text: headings become markdown-style lines, code becomes fenced blocks and tables become pipe-separated rows. The pages mix prose, function signatures, code examples and tables, which makes structure-aware chunking meaningful. The loader also accepts PDF, HTML and text files, so a second document type can

be added without code changes; the results reported here use this one corpus (see Section IX).

B. Preprocessing and Leakage Control

Pages are cleaned before any chunking: scripts, navigation, headers and footers are removed, anchor symbols are dropped, code blocks are wrapped in fences so that they are never cut mid-block, and whitespace is normalised. After cleaning, each document is a single string, and that same string is used for chunking, for locating evidence and for scoring, so character offsets stay valid throughout. Several design choices prevent leakage between the components. Gold evidence is taken from the document itself and is independent of any chunker, so no chunking strategy is advantaged by how the questions were made. The model that splits text for semantic chunking (MiniLM) is used only for splitting and its retrieval scores are not used to build questions. The LLM that drafts questions sees only a text window, never the retrieval systems or their outputs. No hyperparameter was tuned on the gold set: sizes, k and fusion constants were fixed in advance, so the questions act as a pure test set and there is no train–test overlap to control.

C. Gold Question Set

The evaluation set has 85 questions: 60 single-hop, 15 multi-hop and 10 unanswerable. It was drafted with an LLM (Gemini 3.5 Flash-Lite, temperature 0) from randomly sampled windows of 100–260 words that are not mostly code. For a single-hop question the model returns a question, a short answer and one or two sentences copied verbatim from the window as evidence. Evidence is located in the source document by exact string matching, and questions whose evidence cannot be found are discarded, so every answerable question carries character offsets (document, start, end). Multi-hop questions combine two non-adjacent windows of one document and have two evidence spans. Unanswerable questions are plausible questions on the same topic that the window does not answer. The 75 answerable questions are used for retrieval metrics. For answer-level evaluation a fixed random subset of 30 questions, including unanswerable ones, is used per configuration. The prompts are listed in the Appendix. At the time of this report the questions have been drafted and filtered automatically but not yet manually verified (Section IX).

D. Chunking Strategies

All chunkers return character spans into the original document, which is what allows a common evaluation. Table II lists the nine configurations. *Fixed* chunks contain a given number of words with optional overlap. *Sentence* chunks group consecutive sentences (a regular-expression splitter that keeps headings and fenced code as single units). *Semantic* chunking embeds each sentence together with its neighbours (buffer of one) using a small separate model (all-MiniLM-L6-v2), starts a new chunk where the cosine distance between adjacent windows exceeds the 90th percentile, merges chunks under 40 words and caps chunks at 300 words. *Structure-aware* chunking works on paragraph and code blocks and starts a new chunk at a heading (or clause marker such as "Section 5") once the current chunk has at least 40 words, or when adding a block would exceed 300 words; code blocks are never split unless they exceed the limit. *Parent-child* (hierarchical) retrieval embeds 100-word child chunks and returns the enclosing structure-aware section of up to 400 words, de-duplicated across hits.

Chunker	Parameters	Chunks	Avg. words
fixed_128_o0	128 words, no overlap	557	127.1
fixed_128_o32	128 words, 32 overlap	739	127.4
fixed_256_o0	256 words, no overlap	281	251.9
fixed_256_o64	256 words, 64 overlap	370	253.9
sent_4_o1	4 sentences, 1 overlap	1,410	66.2
sent_8_o2	8 sentences, 2 overlap	705	133.1
semantic_p90	90th percentile, 40–300 words	427	165.8
structure_300	headings and blocks, ≤300 words	339	208.8
parent_child	child 100 words, parent ≤400	711	99.6

TABLE II
CHUNKING CONFIGURATIONS AND RESULTING CHUNK SETS (70,791 WORDS)

E. Embedding Models

We compare four open models, all producing 384-dimensional vectors and run locally: BGE-small-en-v1.5 [15] with its query instruction prefix, e5-small-v2 [14] with "query:" and "passage:" prefixes, GTE-small [16] and all-MiniLM-L6-v2 [13]. Inputs are truncated to 512 tokens and vectors are L2-normalised, so inner product equals cosine similarity. Vectors are stored in an exact FAISS flat index [26]. Embeddings are cached on disk so that re-runs are deterministic and fast. The maximum input length is 512 tokens for BGE, E5 and GTE and 256 tokens for MiniLM, so MiniLM silently truncates longer inputs. The chunk sets crossed with all embedders in Stage 2 average 66–133 words, below this limit on average, but individual long chunks (for example those containing code) may still be truncated. API embedders (Gemini, OpenAI) are supported by the code but were not used.

F. Retrieval Pipelines

Dense retrieval returns the top-5 units by cosine similarity. *Hybrid* retrieval runs BM25 [22] over the chunk text in parallel with dense search, takes the top 30 of each and merges them with RRF (k = 60) [23]. *Hybrid + rerank* re-scores the top 20 fused units with the cross-encoder ms-marco-MiniLM-L-6-v2 [24] and returns the best five. For parent-child chunking the retrieved children are mapped to their parents and de-duplicated before fusion output is cut to five units.

G. Evaluation Metrics

Let E be the evidence spans of a question and U the retrieved units, all as character intervals. The coverage of an evidence span e is the share of its characters inside retrieved units, and **context recall** averages it over the spans (so a multi-hop question is rewarded only for finding both pieces):

$$cov(e) = |e \cap \cup_{u \in U} u| \,/\, |e|, \quad R = (1/|E|) \, \Sigma_{e \in E} \, cov(e) \qquad (1)$$

A retrieved unit is relevant if it covers at least 30% of an evidence span or if at least half of the unit lies inside one. **Context precision** is the rank-aware average precision over the m relevant units at ranks $k_1 < \ldots < k_m$ (zero if none are found), and $\mathbf{f}_{ctx}$ is their harmonic mean, our primary ranking metric:

$$P = (1/m) \, \Sigma_{i=1\ldots m} \, i \,/\, k_i, \quad f_{ctx} = 2RP \,/\, (R + P) \qquad (2)$$

We also report Hit@5 (some evidence span at least half covered), MRR, *density* (evidence characters divided by retrieved characters, a measure of noise) and the number of words passed to the generator. Answer quality uses an LLM judge in a single call per question that returns *groundedness* (fraction of claims in the answer supported by the retrieved context), *correctness* (1, 0.5 or 0 against the reference answer; for unanswerable questions 1 only if the system abstains) and an abstention flag. Confidence intervals are 95% percentile bootstrap intervals over questions (2,000 resamples), and differences between configurations use a paired bootstrap on per-question f_{ctx} (5,000 resamples). Latency is the median and 95th percentile of 15 single-query retrievals after a warm-up call, with uncached query encoding. Index size is the stored vector bytes.

H. Worked Example

To make the metrics concrete, consider an illustrative question (numbers constructed for illustration) whose evidence span has 200 characters. Suppose the top-ranked unit has 600 characters and contains 180 of the evidence characters, so $cov = 0.90$ and $R = 0.90$. The unit is relevant and sits at rank 1, so $P = 1$ and $f_{ctx} = 0.947$. If the same unit were ranked third behind two irrelevant units, R would be unchanged but $P = 1/3$, and f_{ctx} falls to 0.486. If a 60-character unit lying inside the evidence were retrieved instead, it would also count as relevant (more than half of it lies in the span), but R could be at most 0.30. The metrics therefore reward finding the evidence, ranking it early and not diluting it with other text, regardless of how the document was cut.

I. Proposed Pipeline

Fig. 1 shows the end-to-end workflow and Table III summarises the components. Everything runs from one notebook in which the LLM provider (Gemini or OpenRouter) is a switchable option and the model is selected from the list available to the user's API key.

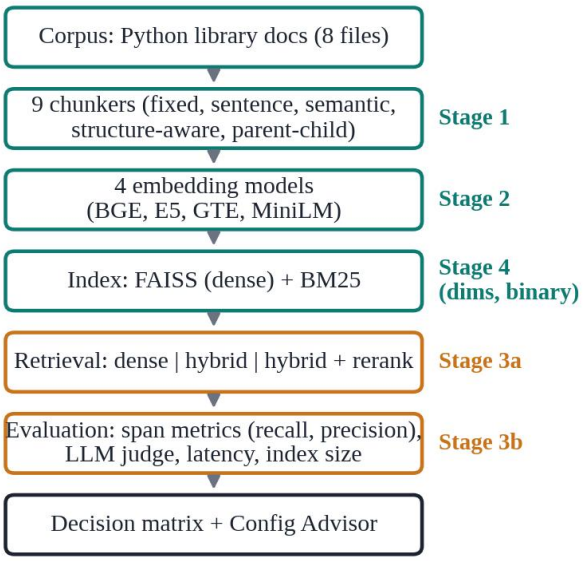


Fig. 1. End-to-end pipeline and the experimental stages (S1–S4) run on it.

Component	Description	Key setting
Corpus	Python standard-library docs as structured text	8 pages, 70,791 words
Gold set	LLM-drafted questions with verbatim evidence spans	60 single, 15 multi, 10 unanswerable
Chunking	Fixed, sentence, semantic, structure-aware, parent-child	9 configurations
Embeddings	Local bi-encoders, cosine over normalised vectors	4 models, 384-d
Retrieval	FAISS flat index, BM25, RRF, cross-encoder	top-5 from 30 candidates
Metrics	Span recall and precision, f_{ctx}, LLM groundedness and correctness	95% bootstrap CIs
LLM	Google AI Studio, temperature 0, responses cached	gemini-3.5-flash-lite
Reproducibility	Single notebook, disk caches, fixed seed	seed 42

TABLE III
COMPACT METHODOLOGY SUMMARY

IV. IMPLEMENTATION

A. Software Architecture

The study is implemented as a single Jupyter notebook organised into the modules of Table IV, so that every experiment can be re-run from a clean environment. Each module is a small set of functions whose only hidden state is an on-disk cache.

Module	Function
Corpus loader	Converts HTML, PDF and text into structured text; keeps one normalised string per document so offsets stay valid
Chunkers	Nine span-based chunkers (words, sentences, semantic, structure-aware, parent-child)
Embedder wrapper	Local sentence-transformers or optional Gemini/OpenAI; query and passage prefixes; truncation; disk cache
Retriever	FAISS flat or binary index, BM25, reciprocal rank fusion, cross-encoder reranking
Gold-set generator	LLM drafting, verbatim-evidence location, CSV export for manual review
Evaluator	Span metrics, LLM judge, latency, index size, bootstrap confidence intervals
LLM layer	Gemini/OpenRouter switch, key-based model picker, retries, SQLite response cache
Reporting	Heatmaps, pipeline plots, decision matrix and the Streamlit Config Advisor

TABLE IV
SOFTWARE MODULES OF THE EXPERIMENT NOTEBOOK

B. LLM Layer and Model Selection

All LLM calls go through one function. The provider is a configuration option (Gemini through Google AI Studio, or OpenRouter), so the same experiment can be repeated with a different generator and judge. Model identifiers change frequently and free-tier models rotate, so none are hard-coded: after the user supplies an API key, the notebook lists the models that key can call (for Gemini, models that support text generation; for OpenRouter, all models with free ones flagged), lets the user choose one from a numbered list or a dropdown, and confirms the choice with a one-word test call. Responses are cached in SQLite under a key made from provider, model, temperature and prompt, so identical calls are never repeated and switching provider never mixes results. Failed calls are retried with exponential back-off and longer waits on rate limiting, with an optional fall-back to the other provider.

C. Prompt Design

The study uses LLMs for three tasks, and each prompt was designed around a specific failure risk. *Question drafting* requires evidence copied verbatim, which lets the pipeline verify by exact string matching that the evidence exists and recover its character offsets; hallucinated or paraphrased evidence is discarded automatically. The prompt also demands self-contained questions that never refer to "the passage", since such questions cannot be retrieved without that context, and a JSON-only output that is parsed defensively. *Generation* restricts the model to the retrieved context and prescribes an exact abstention phrase, which makes abstention detectable and prevents answers from parametric knowledge from inflating groundedness. The *judge* receives context, question, reference and answer and returns three fields in one call, which halves the number of requests compared with separate groundedness and correctness calls, an important saving under free-tier rate limits. All calls use temperature 0. The prompts are given in the Appendix.

D. Reproducibility and Cost Control

Embeddings are cached on disk under a key made from model, prefixes and a hash of the texts, retrievers are cached per (corpus, chunker, embedder, dimension, quantization), and every evaluated configuration is appended to a results file with its stage, settings, provider and model. Before the answer-quality stage the notebook prints the number of LLM requests it will make (about 360 here: two per question, 30 questions, three modes and two pairs), and the gold set is stored as a CSV that can be reviewed and marked as verified, after which only verified rows are used.

V. EXPERIMENTAL SETUP

A. Staged Design

A full grid of nine chunkers, four embedders and three retrieval modes would need 108 configurations before any compression or LLM evaluation. We instead use stages in which each stage narrows the next: **Stage 1** compares all nine chunkers with one fixed embedder (BGE-small, dense). **Stage 2** crosses the three best chunkers with all four embedders. **Stage 3a** runs all nine chunkers with the best Stage 2 embedder (e5-small) in the three retrieval modes, which is the test of H2. **Stage 3b** adds answer generation and judging for the two best (chunker, embedder) pairs in all three modes. **Stage 4** takes the best pair and evaluates truncation to 256, 128 and 64 dimensions and sign-bit binary quantization. In total 62 configuration evaluations were run, and Stages 1–4 and 3a need no LLM calls.

B. Environment and Settings

Experiments ran on a CPU-only machine in Python with sentence-transformers, FAISS (CPU), rank_bm25, pandas and SciPy. The random seed is 42. Answer generation, judging and question drafting used Gemini 3.5 Flash-Lite through Google AI Studio at temperature 0, with responses cached on disk keyed by provider, model and prompt. Retrieval settings were fixed in advance and not tuned: top-5 units, a pool of 30 candidates, a rerank pool of 20, RRF constant 60 and a relevance threshold of 0.3. The reported numbers therefore describe sensible defaults, not the best attainable per method.

C. Primary Metric

We rank configurations by f_{ctx} because the application needs both evidence coverage and low noise: recall alone favours large chunks, and precision alone favours tiny ones. Because f_{ctx} is computed from character overlap, a 66-word sentence group and a 254-word fixed chunk are judged by the same standard.

D. Statistical Procedures

Per-question scores are resampled with replacement to obtain 95% percentile intervals for a configuration mean (2,000 resamples). To compare two configurations on the same questions we resample the per-question differences (5,000 resamples) and report the mean difference, its 95% interval and a two-sided p-value taken as twice the smaller tail proportion of the resampled differences. We regard a comparison as clear only when the interval excludes zero. Only three comparisons were planned in advance for H1 (structure-aware against the best fixed-size, sentence and semantic chunker), so no multiple-comparison correction was applied; all other contrasts in Section VI are descriptive and are labelled as such. Answer-level metrics use 30 questions and are reported without intervals because they are too few for them to be informative.

VI. RESULTS AND DISCUSSION

A. Chunking Strategies (Stage 1)

Table V and Fig. 2 give the Stage 1 results. Fixed-size chunks of 128 words without overlap score highest (f_{ctx} 0.729), closely followed by four-sentence groups (0.722). Semantic chunking is the lowest (0.634), in line with the doubts about its value in [7]. The most important observation is that the 95% intervals overlap heavily, so most differences between chunkers are within noise at this sample size. Structure-aware chunking, which H1 predicted to win, scored 0.675. A paired bootstrap against the best chunker of each other family found no clear difference: −0.054 against fixed-size (CI −0.145 to +0.038, p = 0.239), −0.048 against sentence-based (CI −0.146 to +0.053, p = 0.349) and +0.041 against semantic (CI −0.059 to +0.143, p = 0.388). On this corpus **H1 is therefore not supported**: structure-aware chunking is neither better nor clearly worse than the alternatives.

Chunker	R	P	f_{ctx}	Hit@5	MRR	Dens.	Ctx words
fixed_128_o0	.842	.704	**.729**	.920	.704	.025	638
sent_4_o1	.815	.708	.722	.893	.717	.047	347
sent_8_o2	.855	.664	.706	.907	.684	.025	686
parent_child	.820	.672	.700	.893	.671	.010	1,555
fixed_128_o32	.806	.664	.699	.867	.685	.024	639
fixed_256_o64	.813	.645	.687	.867	.665	.012	1,272
structure_300	.812	.646	.675	.880	.662	.014	1,140
fixed_256_o0	.765	.645	.664	.827	.649	.011	1,267
semantic_p90	.767	.594	.634	.800	.606	.014	999

TABLE V
STAGE 1: CHUNKER COMPARISON (BGE-SMALL, DENSE, TOP-5, 75 QUESTIONS)

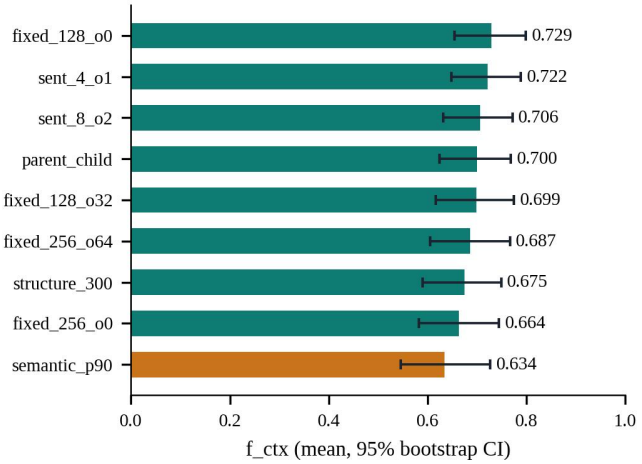


Fig. 2. Stage 1 f_{ctx} per chunker with 95% bootstrap confidence intervals. Semantic chunking (orange) is lowest.

Two trends are visible. First, smaller units scored higher: 128-word fixed chunks reach 0.729 against 0.664 for 256-word chunks without overlap, because precision falls as chunks grow while recall changes little. Second, overlap had no consistent effect: it lowered f_{ctx} for 128-word chunks (0.699 against 0.729) and raised it for 256-word chunks (0.687 against 0.664), and both changes lie inside the intervals.

These comparisons are limited by sample size. With 75 questions the 95% intervals of single configurations are about ±0.08 wide (Table XIII), and the paired intervals above are about ±0.09, so only differences larger than roughly 0.09 in f_{ctx} can be detected reliably. Several differences discussed below (for example 0.729 against 0.722) are therefore within noise, and we treat rankings among the top chunkers as provisional. The conclusions that survive this limit are the coarse ones: semantic chunking is not better, hybrid retrieval helps broadly, and compression below 256 dimensions hurts.

Fig. 3 plots recall against precision. Eight-sentence groups have the highest recall (0.855) but not the highest precision, while four-sentence groups and 128-word chunks balance both. Larger units did not increase recall: 256-word chunks without overlap have the second-lowest recall (0.765), which may be because a single vector represents more text and matches queries less sharply, although we did not test this explanation. Parent-child retrieval returns the largest context but its recall (0.820) is similar to that of much smaller units.

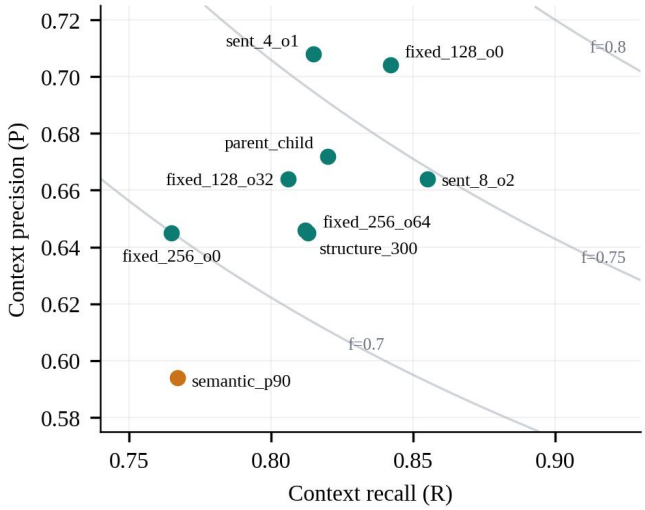


Fig. 3. Stage 1 recall against precision. Curves are constant f_{ctx} contours; semantic chunking (orange) is lowest.

B. Context Cost

Fig. 4 relates quality to the amount of text handed to the generator, which drives answer-generation cost and latency. Four-sentence groups pass only 347 words with a density of 0.047, nearly double that of any other chunker, while 256-word fixed chunks pass about 1,270 words at lower f_{ctx}. Parent-child retrieval passes the most text (1,555 words) because it returns whole sections. Recall is highest for eight-sentence groups (0.855), but at lower precision. If generation cost matters, small sentence-based units are attractive because they achieve similar f_{ctx} with roughly a quarter to a half of the context of larger chunks.

In token terms, using roughly 1.3 tokens per English word as a rule of thumb, four-sentence groups cost about 450 prompt tokens of retrieved context per query, 128-word fixed chunks about 830, 256-word fixed chunks about 1,650 and parent-child retrieval about 2,000. The cheapest setting is thus more than four times cheaper in context tokens than the most expensive one, a saving that applies on every query and scales with traffic, whereas the difference in f_{ctx} between them is small and within noise.

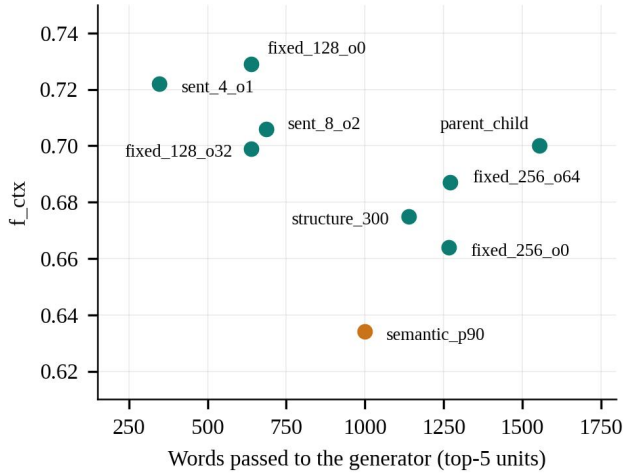


Fig. 4. Quality versus context size (Stage 1). Sentence groups of four sentences give high f_{ctx} with the least context.

C. Embedding Models (Stage 2)

Fig. 5 crosses the three best Stage 1 chunkers with the four embedders. The best embedder depends on the chunker: e5-small is clearly best on four-sentence groups (0.776), gte-small on eight-sentence groups (0.736) and BGE-small and GTE-small tie on fixed 128-word chunks (0.729). The spread across embedders is small for fixed chunks (0.712–0.729) and large for four-sentence groups (0.685–0.776), so the embedding choice matters more for short sentence-level units. MiniLM is lowest in two of three rows. The best pair overall is four-sentence groups with e5-small. This answers part of RQ1: chunker and embedder cannot be chosen independently. Because the three chunkers were chosen with BGE-small in Stage 1, other chunkers were not crossed with all embedders, which we revisit in Section IX.

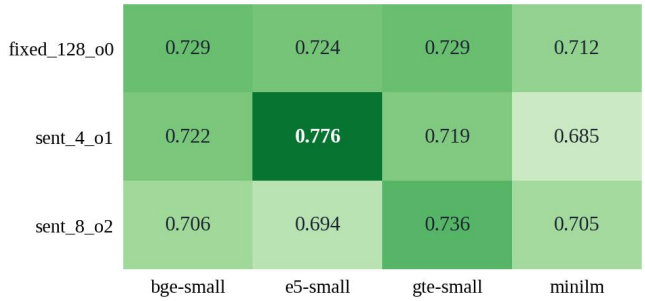


Fig. 5. Stage 2 f_{ctx} for the three best chunkers across four embedders (dense retrieval).

D. Retrieval Pipeline (Stage 3a)

Table VI and Fig. 6 show all nine chunkers with e5-small in the three retrieval modes. Hybrid retrieval improved every chunker, with gains from +0.028 (parent-child) to +0.113 (eight-sentence groups), and raised the mean f_{ctx} from 0.720 to 0.799. Adding the reranker raised the mean only to 0.808 and was inconsistent: it helped five chunkers (largest for parent-child, +0.045) and lowered four (largest for fixed 256-word chunks, −0.029). **H2 is supported in the sense it was stated**: the spread between the best and worst chunker shrinks from 0.095 (dense) to 0.076 (hybrid) to 0.059 (hybrid + rerank). The identity of the best chunker changes with the pipeline (four-sentence groups for dense and hybrid, fixed 128-word chunks with reranking), so picking a winner on differences of 0.01–0.03 is not warranted. The spread is a range statistic over 75 questions and we did not compute an interval for it, so the evidence for H2 is descriptive.

Chunker	Dense	Hybrid	Hyb. + rerank
fixed_128_o0	.724	.802	**.834**
fixed_128_o32	.731	.819	.818
fixed_256_o0	.714	.804	.775
fixed_256_o64	.715	.790	.815
parent_child	.718	.746	.791
semantic_p90	.681	.792	.785
sent_4_o1	**.776**	**.822**	.814
sent_8_o2	.694	.807	.820
structure_300	.728	.808	.822
Mean	.720	.799	.808
Spread (max − min)	.095	.076	.059

TABLE VI
STAGE 3A: f_{ctx} BY CHUNKER AND RETRIEVAL MODE (E5-SMALL)

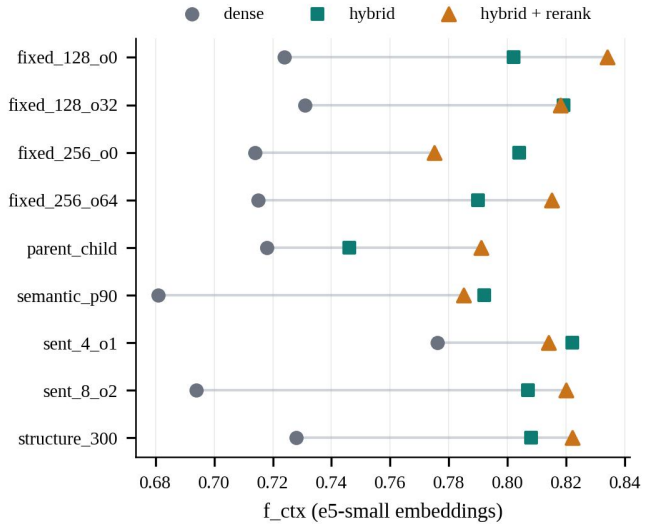


Fig. 6. Stage 3a: f_{ctx} per chunker for dense (circle), hybrid (square) and hybrid + rerank (triangle) retrieval.

Fig. 7 shows the change from each step. The gain from hybrid retrieval is negatively correlated with the dense score (Pearson r = −0.65 over the nine chunkers): hybrid retrieval helped most where dense retrieval was weakest, such as semantic chunks (+0.111) and eight-sentence groups (+0.113), and least for parent-child (+0.028) and four-sentence groups (+0.046). A plausible explanation, which we did not test, is that API documentation contains many exact identifiers (function and parameter names) that BM25 matches reliably even when the dense representation is diluted. The reranker, in contrast, raised f_{ctx} for five chunkers and lowered it for four.

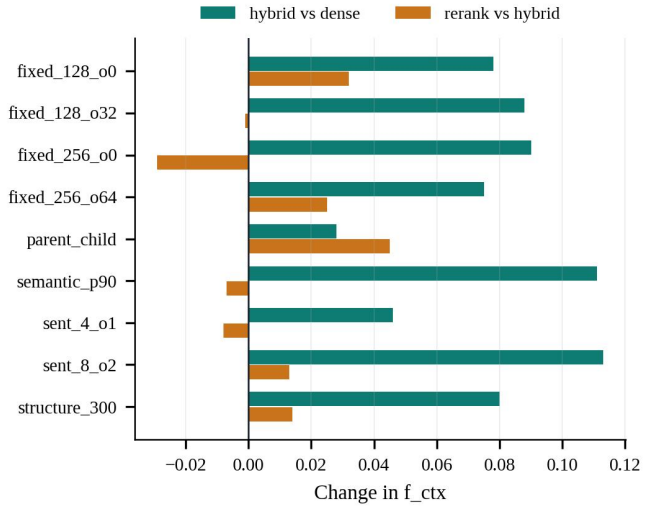


Fig. 7. Change in f_{ctx} from adding BM25 fusion (teal) and then a reranker (orange), per chunker.

E. Answer Quality (Stage 3b)

Table VII reports the LLM-judged results for the two best pairs on a 30-question subset. Groundedness is at or near its ceiling (0.983–1.000), as expected when the generator is told to answer only from the context. Correctness ranges from 0.778 to 0.944. Higher f_{ctx} does not map monotonically to higher correctness: for four-sentence groups with e5-small the dense setting has the best correctness (0.870) although hybrid has the best f_{ctx}, while for eight-sentence groups with gte-small hybrid retrieval gives the best result (0.944). With 30 questions, one question moves correctness by about 0.03, so differences below roughly 0.07 should not be interpreted. The abstention rate is computed on only about three unanswerable questions per configuration (values 0.33 and 0.67), so we do not draw conclusions from it.

Pair	Mode	f_{ctx}	Ground.	Correct.	Abstain
sent_4_o1 + e5	dense	.776	.983	.870	.667
sent_4_o1 + e5	hybrid	.822	1.000	.833	.333
sent_4_o1 + e5	hyb.+rerank	.814	1.000	.815	.667
sent_8_o2 + gte	dense	.736	1.000	.778	.333
sent_8_o2 + gte	hybrid	.807	1.000	**.944**	.333
sent_8_o2 + gte	hyb.+rerank	.822	1.000	.889	.667

TABLE VII
STAGE 3B: ANSWER QUALITY (30 QUESTIONS, GEMINI 3.5 FLASH-LITE JUDGE)

F. Compression and Efficiency (Stage 4)

Table VIII and Fig. 8 evaluate compression for four-sentence groups with e5-small. Truncating to 256 dimensions keeps 96.7% of the full-vector f_{ctx} (0.750 against 0.776) with a one-third smaller index; 128 dimensions keep 86.8% and 64 dimensions only 64.8%. Sign-bit binary quantization reduces the index by a factor of 32 (2.17 MB to 0.068 MB at 384 dimensions) but keeps only 59.3% of quality, and combining it with truncation degrades quality quickly. e5-small was not trained for Matryoshka-style truncation, and the binary codes use no calibration, so these are results for an off-the-shelf small model and not a bound on what compression can achieve. **H3 is therefore only partly supported**: moderate truncation is cheap, binary quantization of this model is not. Median latency stayed between 68 and 73 ms in every setting, which suggests that on a corpus of this size query encoding on CPU, not vector search, dominates the cost; savings from smaller indexes would appear at larger scale.

Dims	Type	f_{ctx}	Rel.	MB	p50 ms	p95 ms
384	float32	**.776**	100%	2.166	70.6	82.2
384	binary	.460	59.3%	0.068	70.1	81.2
256	float32	.750	96.7%	1.444	70.8	124.9
256	binary	.403	52.0%	0.045	69.5	83.2
128	float32	.674	86.8%	0.722	68.3	82.4
128	binary	.217	28.0%	0.023	70.6	84.9
64	float32	.503	64.8%	0.361	68.9	78.5
64	binary	.089	11.5%	0.011	72.6	85.2

TABLE VIII
STAGE 4: COMPRESSION OF E5-SMALL VECTORS (SENT_4_O1, DENSE)

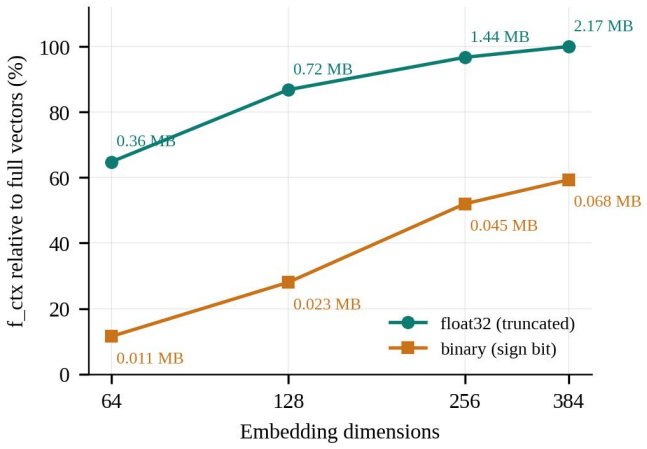


Fig. 8. Stage 4: f_{ctx} relative to full vectors against dimensions, with index size annotated.

VII. DECISION MATRIX AND CONFIG ADVISOR

A. Decision Matrix

To turn the experiments into a recommendation we score every evaluated configuration as a weighted sum of min–max normalised f_{ctx}, normalised (log) median latency and normalised (log) index size, with weights (0.8, 0.1, 0.1) for *accuracy*, (0.4, 0.5, 0.1) for *speed*, (0.4, 0.1, 0.5) for *low cost* and (0.5, 0.25, 0.25) for *balanced*. Table

IX lists the winner per profile. Structure-aware chunking with e5-small and hybrid retrieval wins the accuracy, speed and balanced profiles (f_{ctx} 0.808, recall 0.913, precision 0.778, 26.7 ms, 0.52 MB), and fixed 256-word chunks with e5-small and hybrid retrieval win the low-cost profile (0.804, 0.43 MB). Structure-aware chunking is thus a sensible default for this kind of document even though it did not beat the alternatives in Stage 1: it is competitive under hybrid retrieval and keeps code and sections intact. The reranked fixed-128 configuration has a higher raw f_{ctx} (0.834) but is not selected; because the score also penalises latency and index size, the added cost of reranking is the likely reason, although we did not inspect its per-component score. Latencies are measured on a CPU with 15 queries and are noisy, so the speed profile cannot reliably separate configurations.

Profile	Chunker	Embedder	Mode	f_{ctx}	p50 ms	MB
Accuracy	structure_300	e5-small	hybrid	.808	26.7	0.52
Speed	structure_300	e5-small	hybrid	.808	26.7	0.52
Low cost	fixed_256_o0	e5-small	hybrid	.804	73.8	0.43
Balanced	structure_300	e5-small	hybrid	.808	26.7	0.52

TABLE IX
DECISION MATRIX FOR THE PYTHON DOCUMENTATION CORPUS

B. Config Advisor and Practical Guidelines

The Config Advisor is a small Streamlit application that reads the results file, lets the user choose a document type, sliders for accuracy, latency and storage priorities and allowed embedders and modes, and returns the best-scoring configuration with the top ten alternatives. From the evidence in this study we draw the following cautious guidelines for technical documentation: (i) use hybrid retrieval, which gave the largest and most consistent gain; (ii) prefer small units (about 4–8 sentences or 128 words) when generation cost matters; (iii) do not pay for semantic chunking without evidence on your own data; (iv) choose the embedder after the chunker, since their effects interact; (v) add a reranker only after checking it on your documents, since its effect varied by chunker; (vi) truncate to about 256 dimensions if storage matters, but avoid naive binary codes with small models.

VIII. SYNTHESIS: RESEARCH QUESTIONS AND PRIOR WORK

A. Answers to the Research Questions

RQ1 (dependence of the best combination on the document and pipeline). Within this corpus the best choice depends on the neighbouring components: the best embedder changes with the chunker (e5-small for four-sentence groups, gte-small for eight-sentence groups) and the best chunker changes with the retrieval mode. Dependence on the *document type* could not be tested with one corpus. **RQ2** (does a stronger pipeline reduce sensitivity to chunking). Yes, in the sense of spread: 0.095, 0.076 and 0.059 for dense, hybrid and hybrid with reranking. Most of the benefit comes from hybrid retrieval, and the reranker's effect is inconsistent. **RQ3** (cost of compression). Truncation to 256 dimensions is almost free (96.7% retained) while 128 dimensions or fewer, and sign-bit binary codes of this model, lose substantial quality. Table X summarises the hypotheses.

	Hypothesis	Evidence in this study	Verdict
H1	Structure-aware beats fixed-size and semantic chunking	structure_300 0.675 against 0.729 (fixed), 0.722 (sentence), 0.634 (semantic); paired CIs include zero	Not supported
H2	A reranker narrows the gap between chunkers	Spread 0.095 → 0.076 → 0.059 across dense, hybrid, hybrid + rerank; no CI for spread	Supported (descriptive)
H3	Truncated or binary embeddings lose little accuracy	256-d keeps 96.7%; binary keeps 59.3% at 384-d, 52.0% at 256-d	Partly supported

TABLE X
SUMMARY OF HYPOTHESES

B. Relation to Prior Work

Our result that semantic chunking did not pay off agrees with Qu et al. [7]. Yepes et al. [9] report gains from structure-aware chunking on financial reports; we did not observe them on API documentation, but their domain and baselines differ and our intervals are wide (roughly ±0.08), so the studies are not in conflict so much as inconclusive for our setting. Our finding that hybrid retrieval gave the most reliable improvement is consistent with the recommendation of balanced component combinations in [25]. The benchmark-based reports on binary quantization [21] describe much of retrieval quality being retained, whereas we measured 59% for a small model with uncalibrated sign-bit codes; differences in model size, calibration and evaluation setting may explain this. Finally, [28] warns of judge biases, which is why we kept the judge out of the retrieval metrics.

C. Generalisability

Not every finding is equally likely to transfer to other documents. The result that hybrid retrieval helps most where dense retrieval is weak is plausibly general, because it rests on the complementary strengths of lexical and semantic matching, though our explanation about exact identifiers is untested and would be weaker for prose-heavy corpora. The interaction between chunker and embedder is a reason to re-tune rather than a result to copy. The chunker ranking itself is the least transferable finding, since differences were within noise and the corpus is a single API reference. The compression results are specific to one small, non-Matryoshka model. We therefore present the decision matrix as a worked procedure, backed by a reproducible notebook, to be re-run on a target corpus, and not as a universal recommendation.

IX. LIMITATIONS AND FUTURE WORK

Single corpus. All results come from the Python documentation, so H1 and the document-type part of RQ1 are not tested across document types; the pipeline already accepts legal or regulatory text and this is the first extension. **Unverified questions.** The 85 questions were drafted by an LLM and filtered by exact evidence matching but not yet manually verified. A question may be answerable from other passages (this affects all configurations equally) and some may be ambiguous, so absolute numbers may shift after review. **Small samples.** Retrieval metrics use 75 questions, answer metrics 30 and the abstention rate about three, so differences of a few hundredths are not meaningful. **Staged selection.** Stage 2 only crossed the top three chunkers from a BGE-small ranking, so a chunker that suits another embedder better may have been missed. **No tuning.** Chunk sizes, k and fusion parameters were fixed. **Judge.** The same LLM family drafted questions and judged answers, which risks self-preference [28]; a second-provider check is implemented but has not been run. **Latency.** Measurements are CPU-bound with 15 queries. **Scoring.** Min–max normalisation in the decision matrix includes deliberately degraded compressed configurations in its range, which compresses accuracy differences. **Compression.** Only one non-Matryoshka model was compressed, with uncalibrated binary codes.

Responsible use. Only public documentation was sent to the LLM provider. The free tier of a hosted LLM service may retain prompts to improve its models, so private or confidential documents should not be used with it; local models or a paid tier with data controls are the safer choice for such corpora. All embeddings in this study were computed locally.

Future work includes: (1) adding a legal or regulatory corpus and re-testing H1 across document types; (2) manually verifying the question set and re-running all stages with confidence intervals for the spread statistic; (3) running the cross-provider judge check and a larger reranker such as BGE-reranker; (4) extending the chunker set with late chunking and contextual retrieval and crossing all chunkers with all embedders; and (5) evaluating Matryoshka-trained models and calibrated quantization for compression.

Planned work	Purpose	Expected output
Legal or regulatory corpus	Test H1 and RQ1 across document types	Per-corpus decision matrices
Manual verification of questions	Remove ambiguous or multi-answer items	Verified benchmark, refreshed results
Second-provider judge	Check robustness to the judge	Rank agreement of configurations
Larger reranker, more chunkers	Test late and contextual chunking	Extended Stage 3 comparison
Matryoshka and calibrated codes	Re-test H3 with suitable models	Fair compression results

TABLE XI
PLANNED EXTENSIONS

X. CONCLUSION

We compared nine chunking strategies, four embedding models and three retrieval pipelines on technical documentation with a chunk-size-independent, span-based evaluation, and distilled the results into a decision matrix and an advisor tool. On the Python documentation corpus, most chunkers were statistically indistinguishable and semantic chunking was lowest; the best embedder depended on the chunker; hybrid retrieval gave the largest consistent improvement; reranking narrowed the gap between chunkers but did not help every one; and 256-dimensional embeddings retained 97% of quality while naive binary quantization did not. The practical message is that the retrieval pipeline and the embedder matter at least as much as the chunking strategy, and that recommendations should be checked on one's own documents. Because the evidence comes from one corpus and an unverified question set, these findings are preliminary and the planned extensions in Section IX are needed before they can be generalised.

APPENDIX

A. Prompts (verbatim)

Question drafting, single-hop: You are building a retrieval test set. Read the PASSAGE and write ONE specific question that can be answered using only this passage. The question must be self-contained (never say 'the passage' or 'the text'). Also give the short answer and copy, EXACTLY and verbatim from the passage, the one or two consecutive sentences that contain the answer. Return ONLY JSON: {"question": "...", "answer": "...", "evidence": "..."}. PASSAGE: <text>

Question drafting, multi-hop: You are building a retrieval test set. Write ONE self-contained question that can only be answered by combining information from BOTH passages below (never say 'passage'). Give the short answer, and copy EXACTLY and verbatim one or two consecutive sentences from each passage that are needed. Return ONLY JSON: {"question": "...", "answer": "...", "evidence_a": "...", "evidence_b": "..."}. PASSAGE A: <text> PASSAGE B: <text>

Question drafting, unanswerable: Read the PASSAGE and write ONE plausible, self-contained question on the same topic that the passage does NOT answer (it must not be answerable from the passage). Never say 'the passage'. Return ONLY JSON: {"question": "..."}. PASSAGE: <text>

Generation: Answer the question using ONLY the context below. If the context does not contain the answer, reply exactly: I don't know. Context: <numbered retrieved units> Question: <question> Answer:

Judge: You are a strict evaluator. Return ONLY JSON with keys groundedness, correctness, abstained. groundedness: fraction (0 to 1) of the factual claims in ANSWER that are supported by CONTEXT (1.0 if the answer only says it does not know). correctness: 1 if ANSWER matches REFERENCE in substance, 0.5 if partially, else 0; if REFERENCE is UNANSWERABLE, correctness is 1 only if the ANSWER abstains, else 0. abstained: true if the ANSWER says it does not know. CONTEXT: <context> QUESTION: <question> REFERENCE: <reference> ANSWER: <answer>

B. Fixed Settings and Bootstrap Intervals

Parameter	Value
Units returned (k)	5
Candidate pool / rerank pool	30 / 20
RRF constant	60
Relevance threshold	30% of an evidence span, or half of the unit
Chunker for semantic splitting	all-MiniLM-L6-v2, buffer 1, 90th percentile
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2
Embedder in Stage 1	BGE-small-en-v1.5
Gold questions	60 single-hop, 15 multi-hop, 10 unanswerable
LLM subset per configuration	30 questions
Latency queries	15 (after one warm-up)
Bootstrap resamples	2,000 (CI), 5,000 (paired)
Random seed	42

TABLE XII
FIXED EXPERIMENT SETTINGS

Chunker (Stage 1)	f_{ctx}	95% bootstrap CI
fixed_128_o0	.729	[.655, .799]
sent_4_o1	.722	[.648, .789]
sent_8_o2	.706	[.632, .772]
parent_child	.700	[.624, .769]
fixed_128_o32	.699	[.617, .775]
fixed_256_o64	.687	[.605, .767]
structure_300	.675	[.590, .749]
fixed_256_o0	.664	[.583, .745]
semantic_p90	.634	[.546, .727]

TABLE XIII
STAGE 1 f_{ctx} WITH 95% BOOTSTRAP INTERVALS

C. Scoring Profiles of the Decision Matrix

Profile	Accuracy	Latency	Index size
Accuracy	0.8	0.1	0.1
Speed	0.4	0.5	0.1
Low cost	0.4	0.1	0.5
Balanced	0.5	0.25	0.25

TABLE XIV
WEIGHTS OF f_{ctx}, LATENCY AND INDEX SIZE IN EACH PROFILE

D. Reproduction Steps

1) Place documents in a folder per corpus (data/tech_docs, data/legal); the technical corpus is downloaded automatically.
2) Store the provider API key in a .env file and run the model picker to choose a model available to that key.
3) Run the gold-set generator, open the CSV for each corpus, correct or delete weak questions and mark good ones as verified.
4) Run Stages 1–4 in order; each stage reads the previous stage's ranking and appends rows to the results file.
5) Run the statistics, plotting and decision-matrix cells; run the Streamlit app to query the recommendations.

E. Abbreviations

RAG: retrieval-augmented generation. LLM: large language model. BM25: Okapi best-matching 25 ranking function. RRF: reciprocal rank fusion. FAISS: Facebook AI Similarity Search. CI: confidence interval. MRR: mean reciprocal rank. R, P: context recall and precision. f_{ctx}: harmonic mean of R and P. Hit@5: some evidence span at least half covered in the top five units. Density: evidence characters divided by retrieved characters. p50, p95: median and 95th percentile latency.

ACKNOWLEDGMENT

The authors thank the course instructors for guidance during the reviews, and the open-source communities behind sentence-transformers, FAISS, rank_bm25 and the Python documentation used in this work.

DATA AND CODE AVAILABILITY

The experiment notebook, the gold question files (before and after review), the results file, the decision matrix and the Config Advisor application accompany this report as project artefacts. All embedding models and the reranker used are openly available; LLM outputs are cached so that reported numbers can be reproduced without new calls.

REFERENCES

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[2] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020.

[3] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[4] S. Barnett et al., "Seven failure points when engineering a retrieval augmented generation system," in *Proc. IEEE/ACM CAIN*, 2024, arXiv:2401.05856.

[5] N. F. Liu et al., "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, 2024.

[6] F. Cuconasu et al., "The power of noise: Redefining retrieval for RAG systems," in *Proc. ACM SIGIR*, 2024.

[7] R. Qu, R. Tu, and F. Bao, "Is semantic chunking worth the computational cost?" in *Findings of NAACL*, 2025, arXiv:2410.13070.

[8] T. Chen et al., "Dense X retrieval: What retrieval granularity should we use?" in *Proc. EMNLP*, 2024, arXiv:2312.06648.

[9] A. J. Yepes et al., "Financial report chunking for effective retrieval augmented generation," arXiv:2402.05131, 2024.

[10] P. Finardi et al., "The chronicles of RAG: The retriever, the chunk and the generator," arXiv:2401.07883, 2024.

[11] M. Günther et al., "Late chunking: Contextual chunk embeddings using long-context embedding models," arXiv:2409.04701, 2024.

[12] Anthropic, "Introducing Contextual Retrieval," Anthropic blog, Sep. 2024.

[13] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019.

[14] L. Wang et al., "Text embeddings by weakly-supervised contrastive pre-training," arXiv:2212.03533, 2022.

[15] S. Xiao et al., "C-Pack: Packed resources for general Chinese embeddings," in *Proc. ACM SIGIR*, 2024, arXiv:2309.07597.

[16] Z. Li et al., "Towards general text embeddings with multi-stage contrastive learning," arXiv:2308.03281, 2023.

[17] Z. Nussbaum et al., "Nomic Embed: Training a reproducible long context text embedder," arXiv:2402.01613, 2024.

[18] N. Muennighoff et al., "MTEB: Massive text embedding benchmark," in *Proc. EACL*, 2023.

[19] N. Thakur et al., "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. NeurIPS Datasets and Benchmarks*, 2021.

[20] A. Kusupati et al., "Matryoshka representation learning," in *Proc. NeurIPS*, 2022.

[21] A. Shakir, T. Aarsen, and S. Lee, "Binary and scalar embedding quantization for significantly faster and cheaper retrieval," Hugging Face blog, Mar. 2024.

[22] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, 2009.

[23] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. ACM SIGIR*, 2009.

[24] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[25] X. Wang et al., "Searching for best practices in retrieval-augmented generation," in *Proc. EMNLP*, 2024.

[26] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, 2021.

[27] S. Es et al., "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. EACL (Demos)*, 2024.

[28] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Proc. NeurIPS Datasets and Benchmarks*, 2023.

[29] Python Software Foundation, "The Python Standard Library," Python 3 documentation, docs.python.org/3/library/, accessed 2026.